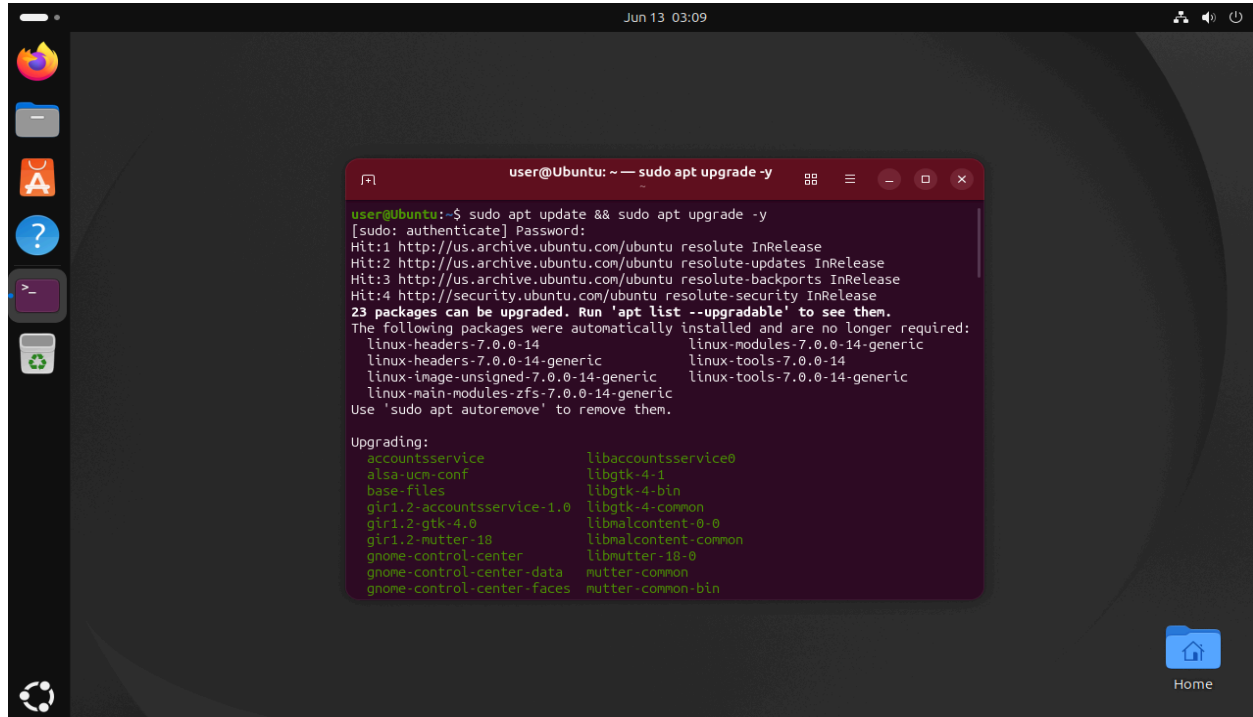


Linux System Administration & Network Analysis Lab — [6/12 for Linux and 6/13 for Wireshark]

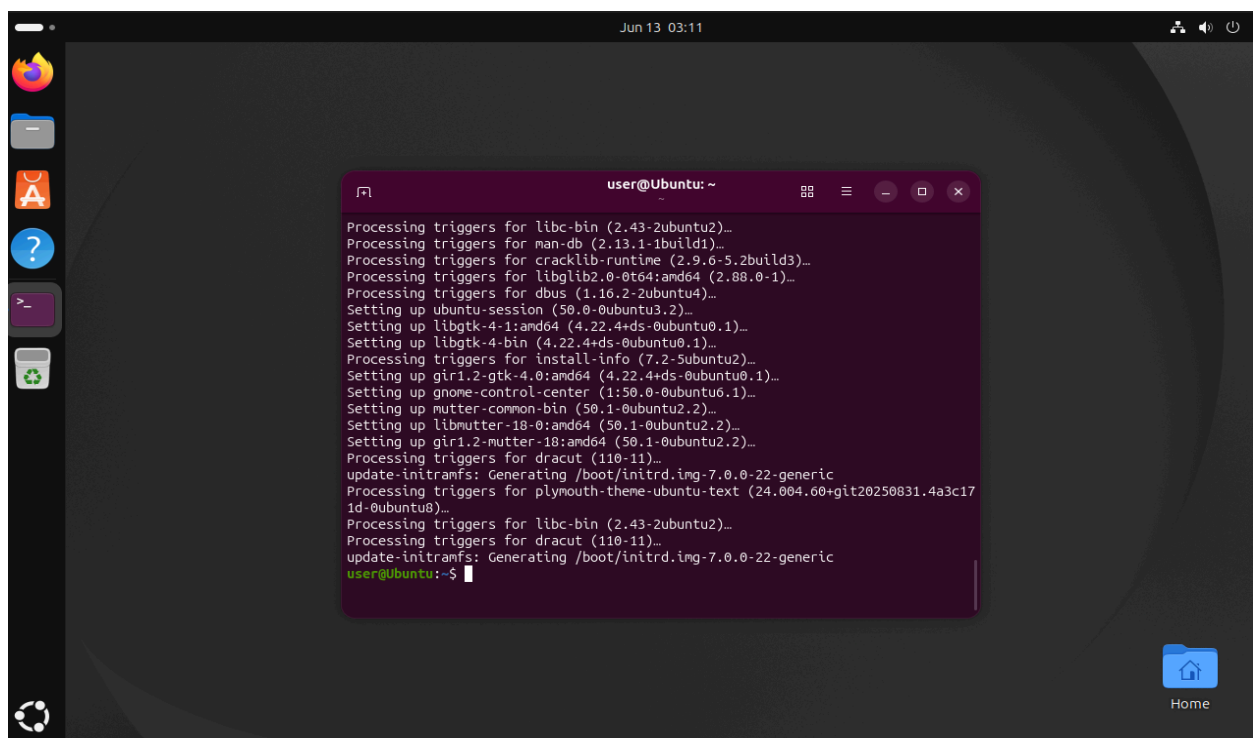
OBJECTIVE: Build and administer a Linux VM to practice sysadmin fundamentals

ENVIRONMENT: Ubuntu 26.04 LTS on VirtualBox



```
user@Ubuntu: ~ — sudo apt upgrade -y
user@Ubuntu:~$ sudo apt update && sudo apt upgrade -y
[sudo: authenticate] Password:
Hit:1 http://us.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
23 packages can be upgraded. Run 'apt list --upgradable' to see them.
The following packages were automatically installed and are no longer required:
linux-headers-7.0.0-14 linux-modules-7.0.0-14-generic
linux-headers-7.0.0-14-generic linux-tools-7.0.0-14
linux-image-unsigned-7.0.0-14-generic linux-tools-7.0.0-14-generic
linux-main-modules-zfs-7.0.0-14-generic
Use 'sudo apt autoremove' to remove them.

Upgrading:
accountsservice libaccountsservice0
alsa-ucm-conf libgtk-4-1
base-files libgtk-4-bin
gir1.2-accountsservice-1.0 libgtk-4-common
gir1.2-gtk-4.0 libnmacontent-0-0
gir1.2-mutter-18 libnmacontent-common
gnome-control-center libmutter-18-0
gnome-control-center-data mutter-common
gnome-control-center-faces mutter-common-bin
```



```
user@Ubuntu: ~
Processing triggers for libc-bin (2.43-2ubuntu2)...
Processing triggers for man-db (2.13.1-1build1)...
Processing triggers for cracklib-runtime (2.9.6-5.2build3)...
Processing triggers for libgl2.0-0:amd64 (2.88.0-1)...
Processing triggers for dbus (1.16.2-2ubuntu4)...
Setting up ubuntu-session (50.0-0ubuntu3.2)...
Setting up libgtk-4-1:amd64 (4.22.4+ds-0ubuntu0.1)...
Setting up libgtk-4-bin (4.22.4+ds-0ubuntu0.1)...
Processing triggers for install-info (7.2-5ubuntu2)...
Setting up gir1.2-gtk-4.0:amd64 (4.22.4+ds-0ubuntu0.1)...
Setting up gnome-control-center (1:50.0-0ubuntu6.1)...
Setting up mutter-common-bin (50.1-0ubuntu2.2)...
Setting up libmutter-18-0:amd64 (50.1-0ubuntu2.2)...
Setting up gir1.2-mutter-18:amd64 (50.1-0ubuntu2.2)...
Processing triggers for dracut (110-11)...
update-initramfs: Generating /boot/initrd.img-7.0.0-22-generic
Processing triggers for plymouth-theme-ubuntu-text (24.004.60+git20250831.4a3c171d-0ubuntu8)...
Processing triggers for libc-bin (2.43-2ubuntu2)...
Processing triggers for dracut (110-11)...
update-initramfs: Generating /boot/initrd.img-7.0.0-22-generic
user@Ubuntu:~$
```

- Installation Notes: There were no installation issues

SECTION 1: USER MANAGEMENT

- Commands used:

```
sudo adduser testuser
sudo adduser adminuser
getent passwd testuser
getent passwd adminuser
sudo groupadd engineers
sudo usermod -aG engineers testuser
sudo usermod -aG engineers adminuser
getent group engineers
id testuser
id adminuser
cat /etc/group | grep engineers
```

- getent output for testuser/adminuser:

```
user@Ubuntu:~$ getent passwd testuser
testuser:x:1001:1001:Test1,1,123,321,N/A:/home/testuser:/bin/bash
user@Ubuntu:~$ getent passwd adminuser
adminuser:x:1002:1002:Admin1,2,456,654,N//A:/home/adminuser:/bin/bash
```

- Group membership verified via:

```
user@Ubuntu:~$ getent group engineers
engineers:x:1003:testuser,adminuser
user@Ubuntu:~$ id testuser
uid=1001(testuser) gid=1001(testuser) groups=1001(testuser),100(users),1003(engineers)
user@Ubuntu:~$ id adminuser
uid=1002(adminuser) gid=1002(adminuser) groups=1002(adminuser),100(users),1003(engineers)
user@Ubuntu:~$ cat /etc/group | grep engineers
engineers:x:1003:testuser,adminuser
```

- What I learned:

I learned that `getent` is the proper `sysadmin` command for querying system databases. It is more reliable than directly reading `/etc/passwd` because it works across different authentication systems. I also learned how Linux groups can be used to manage shared access and that `usermod -aG` adds users to a group without removing existing group memberships. The `id` command confirmed that both users were successfully added to the `engineers` group. The `getent group engineers` verified the group's membership within the system database. Output format is `username:password:UID:GID:info:home:shell`.

SECTION 2: PERMISSIONS

- Commands used:

```
mkdir ~/public_folder
mkdir ~/private_folder
chmod 755 ~/public_folder
chmod 700 ~/private_folder
touch ~/public_folder/testfile.txt
sudo chown testuser:engineers ~/public_folder/testfile.txt
ls -la ~/
ls -la ~/public_folder/
```

- ls -la outputs:

```
user@Ubuntu:~$ ls -la ~/
total 84
drwxr-x--- 18 user user 4096 Jun 13 03:32 .
drwxr-xr-x  5 root root 4096 Jun 13 03:14 ..
-rw-r--r--  1 user user  220 Feb 13 12:16 .bash_logout
-rw-r--r--  1 user user 3771 Feb 13 12:16 .bashrc
drwx----- 13 user user 4096 Jun 13 03:05 .cache
drwxr-xr-x 14 user user 4096 Jun 13 03:03 .config
drwx-----  2 user user 4096 Jun 13 03:05 .gnupg
drwx-----  4 user user 4096 Jun 13 02:56 .local
-rw-r--r--  1 user user  807 Feb 13 12:16 .profile
drwx-----  2 user user 4096 Jun 13 02:57 .ssh
drwxr-xr-x  2 user user 4096 Jun 13 02:56 Desktop
drwxr-xr-x  2 user user 4096 Jun 13 02:56 Documents
drwxr-xr-x  2 user user 4096 Jun 13 02:56 Downloads
drwxr-xr-x  2 user user 4096 Jun 13 02:56 Music
drwxr-xr-x  2 user user 4096 Jun 13 02:56 Pictures
drwxr-xr-x  2 user user 4096 Jun 13 02:56 Public
drwxr-xr-x  2 user user 4096 Jun 13 02:56 Templates
drwxr-xr-x  2 user user 4096 Jun 13 02:56 Videos
drwx-----  2 user user 4096 Jun 13 03:32 private_folder
drwxr-xr-x  2 user user 4096 Jun 13 03:33 public_folder
drwx-----  5 user user 4096 Jun 13 03:00 snap
user@Ubuntu:~$ ls -la ~/public_folder/
total 8
drwxr-xr-x  2 user user 4096 Jun 13 03:33 .
drwxr-x--- 18 user user 4096 Jun 13 03:32 ..
-rw-rw-r--  1 testuser engineers  0 Jun 13 03:33 testfile.txt
user@Ubuntu:~$ ls -la ~/private_folder/
total 8
drwx-----  2 user user 4096 Jun 13 03:32 .
drwxr-x--- 18 user user 4096 Jun 13 03:32 ..
```

- What I learned about the Principle of Least Privilege:

I learned that Linux implements this principle through file permissions. The *public_folder* was configured with 755 permissions, which allows the owner full access (read, write, execute), while group and others can only read/execute. The *private_folder* was configured with 700 permissions, which restricts access entirely to the owner. The `chown` command was used to assign ownership of *testfile.txt* to the user *testuser* and group *engineers*.

SECTION 3: SERVICES

- Commands used:

```
sudo apt install openssh-server -y
sudo systemctl status ssh
sudo systemctl list-unit-files | grep ssh
sudo systemctl stop ssh
sudo systemctl start ssh
sudo systemctl restart ssh
sudo systemctl enable ssh
sudo systemctl is-enabled ssh
sudo systemctl list-unit-files | grep ssh
ssh localhost
```

- SSH status observed:

```
user@Ubuntu:~$ sudo systemctl start ssh
[sudo: authenticate] Password:
user@Ubuntu:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-06-13 04:00:36 UTC; 36min ago
     Invocation: c807ee39423944d2821cf691dcc73f54
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
    Main PID: 19927 (sshd)
      Tasks: 1 (limit: 1839)
     Memory: 1.3M (peak: 5.9M)
        CPU: 42ms
    CGroup: /system.slice/ssh.service
            └─19927 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jun 13 04:00:36 Ubuntu systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Jun 13 04:00:36 Ubuntu sshd[19927]: Server listening on 0.0.0.0 port 22.
Jun 13 04:00:36 Ubuntu sshd[19927]: Server listening on :: port 22.
Jun 13 04:00:36 Ubuntu systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Jun 13 04:08:59 Ubuntu sshd-session[20414]: Accepted password for user from 127.0.0.1 port 47698 ssh2
Jun 13 04:08:59 Ubuntu sshd-session[20414]: pam_unix(sshd:session): session opened for user user(uid=1000) by user(uid=0)
```

```

user@Ubuntu:~$ sudo systemctl list-unit-files | grep ssh
ssh.service                                enabled          enabled
sshd-keygen.service                       enabled          enabled
sshd-unix-local@.service                  alias            -
sshd.service                              alias            -
sshd@.service                             indirect         enabled
sssd-ssh.service                         indirect         enabled
ssh.socket                                enabled          enabled
sshd-unix-local.socket                   generated        -
sssd-ssh.socket                           enabled          enabled
rescue-ssh.target                         static           -
ssh-access.target                         static           -

```

- SSH connectivity:

```

user@Ubuntu:~$ ssh localhost
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is: SHA256:pdzHrE3CdUcgDqZsagla13V5KguFoPB+lERZBxfV2No
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
user@localhost's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-22-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

user@Ubuntu:~$ exit
logout
Connection to localhost closed.

```

- What I learned:

I learned that systemd services can exist in different states. A service that is running is currently active in memory. An enabled service is configured to automatically start during system boot. I used *systemctl* to start, stop, restart, and verify the status of the SSH service. I also learned that SSH is used for secure remote administration of Linux systems. SSH connectivity was successfully configured and tested by connecting to the local host using OpenSSH.

SECTION 4: LOGS

- Commands used:

```
journalctl
journalctl -n 50
journalctl -u ssh
sudo cat /var/log/auth.log
sudo tail -50 /var/log/auth.log
sudo tail -f /var/log/auth.log
last
sudo apt install wtmpdb -y
```

- What I found in auth.log:

```
user@Ubuntu:~$ sudo tail -f /var/log/auth.log
2026-06-13T04:49:37.720749+00:00 Ubuntu sudo: pam_unix(sudo:session): session opened for user root(uid=0) by user(uid=1000)
2026-06-13T04:49:37.814991+00:00 Ubuntu sudo: pam_unix(sudo:session): session closed for user root
2026-06-13T04:50:02.029632+00:00 Ubuntu sudo: pam_unix(sudo:session): session opened for user root(uid=0) by user(uid=1000)
2026-06-13T04:50:02.029819+00:00 Ubuntu sudo: user : TTY=/dev/pts/0 ; PWD=/home/user ; USER=root ; COMMAND=/usr/bin/apt update
2026-06-13T04:50:04.309898+00:00 Ubuntu sudo: pam_unix(sudo:session): session closed for user root
2026-06-13T04:52:12.517238+00:00 Ubuntu sudo: pam_unix(sudo:session): session opened for user root(uid=0) by user(uid=1000)
2026-06-13T04:52:12.517460+00:00 Ubuntu sudo: user : TTY=/dev/pts/0 ; PWD=/home/user ; USER=root ; COMMAND=/usr/bin/tail -50 /var/log/auth.log
2026-06-13T04:52:12.521372+00:00 Ubuntu sudo: pam_unix(sudo:session): session closed for user root
2026-06-13T04:52:41.546524+00:00 Ubuntu sudo: pam_unix(sudo:session): session opened for user root(uid=0) by user(uid=1000)
2026-06-13T04:52:41.546641+00:00 Ubuntu sudo: user : TTY=/dev/pts/0 ; PWD=/home/user ; USER=root ; COMMAND=/usr/bin/tail -f /var/log/auth.log
```

- What journalctl -u ssh showed:

```
user@Ubuntu:~$ journalctl -u ssh
Jun 13 04:00:23 Ubuntu systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Jun 13 04:00:23 Ubuntu sshd[19913]: Server listening on 0.0.0.0 port 22.
Jun 13 04:00:23 Ubuntu sshd[19913]: Server listening on :: port 22.
Jun 13 04:00:23 Ubuntu systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Jun 13 04:00:36 Ubuntu systemd[1]: Stopping ssh.service - OpenBSD Secure Shell server...
Jun 13 04:00:36 Ubuntu sshd[19913]: Received signal 15; terminating.
Jun 13 04:00:36 Ubuntu systemd[1]: ssh.service: Deactivated successfully.
Jun 13 04:00:36 Ubuntu systemd[1]: Stopped ssh.service - OpenBSD Secure Shell server.
Jun 13 04:00:36 Ubuntu systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Jun 13 04:00:36 Ubuntu sshd[19927]: Server listening on 0.0.0.0 port 22.
Jun 13 04:00:36 Ubuntu sshd[19927]: Server listening on :: port 22.
Jun 13 04:00:36 Ubuntu systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Jun 13 04:08:59 Ubuntu sshd-session[20414]: Accepted password for user from 127.0.0.1 port 47698 ssh2
Jun 13 04:08:59 Ubuntu sshd-session[20414]: pam_unix(sshd:session): session opened for user user(uid=1000) by user(uid=0)
Jun 13 04:38:32 Ubuntu systemd[1]: Stopping ssh.service - OpenBSD Secure Shell server...
Jun 13 04:38:32 Ubuntu sshd[19927]: Received signal 15; terminating.
Jun 13 04:38:32 Ubuntu systemd[1]: ssh.service: Deactivated successfully.
Jun 13 04:38:32 Ubuntu systemd[1]: Stopped ssh.service - OpenBSD Secure Shell server.
```

- What last showed:

```
user@Ubuntu:~$ last
user      pts/1      127.0.0.1    Sat Jun 13 04:09 - still logged in
user      tty2      local        Sat Jun 13 02:56 - still logged in
```

- What I learned:

I learned that *journalctl* is used to review service logs generated by the SSH service. The *auth.log* file records every login attempt, sudo usage, and authentication event on the system. The SSH journal showed service startup and shutdown events, successful user authentication attempts, and SSH daemon activity. I also used the *last* command to review login history and identify active user sessions. It displayed active user sessions, login times, and connection sources. Entries show pam_unix opening and closing root sessions, which confirms that sudo commands were logged correctly. I also learned that Ubuntu 26.04 uses the wtmpdb package to provide login history functionality.

SECTION 5: NETWORKING

- Commands used:

```
hostnamectl
ip a
hostname -I
ip r
ping google.com -c 4
nslookup google.com
dig google.com
sudo ss -tulnp
sudo apt install net-tools -y
sudo netstat -tulnp
```

- VM IP Address:

```
user@Ubuntu:~$ hostname -I
10.0.2.15 fd17:625c:f037:2:a00:27ff:fe47:d601
```

- Gateway (10.0.2.2) / Network Interface (enp0s3):

```
user@Ubuntu:~$ ip r
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
```

- Open ports:

```

user@Ubuntu:~$ sudo ss -tulnp
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
udp UNCONN 0 0 0.0.0.0:5353 0.0.0.0:* users:(("avahi-daemon",pid=1428,fd=12))
udp UNCONN 0 0 10.0.2.15:3702 0.0.0.0:* users:(("python3",pid=6536,fd=10))
udp UNCONN 0 0 239.255.255.250:3702 0.0.0.0:* users:(("python3",pid=6536,fd=8))
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:* users:(("systemd-resolve",pid=678,fd=18))
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:* users:(("systemd-resolve",pid=678,fd=16))
udp UNCONN 0 0 127.0.0.1:323 0.0.0.0:* users:(("chronyd",pid=1890,fd=4))
udp UNCONN 0 0 0.0.0.0:56006 0.0.0.0:* users:(("python3",pid=6536,fd=9))
udp UNCONN 0 0 [::]:5353 [::]:* users:(("avahi-daemon",pid=1428,fd=13))
udp UNCONN 0 0 [fe80::a00:27ff:fe47:d601]%enp0s3:3702 [::]:* users:(("python3",pid=6536,fd=13))
udp UNCONN 0 0 [ff02::c]%enp0s3:3702 [::]:* users:(("python3",pid=6536,fd=11))
udp UNCONN 0 0 *:48816 *:~ users:(("python3",pid=6536,fd=12))
udp UNCONN 0 0 [::]:323 [::]:* users:(("chronyd",pid=1890,fd=5))
tcp LISTEN 0 4096 127.0.0.1:631 0.0.0.0:* users:(("cupsd",pid=1764,fd=7))
tcp LISTEN 0 4096 127.0.0.54:53 0.0.0.0:* users:(("systemd-resolve",pid=678,fd=19))
tcp LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:* users:(("systemd-resolve",pid=678,fd=17))
tcp LISTEN 0 4096 0.0.0.0:22 0.0.0.0:* users:(("systemd",pid=1,fd=190))
tcp LISTEN 0 4096 [::]:631 [::]:* users:(("cupsd",pid=1764,fd=6))
tcp LISTEN 0 4096 [::]:22 [::]:* users:(("systemd",pid=1,fd=191))

user@Ubuntu:~$ sudo netstat -tulnp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address Foreign Address State PID/Program name
tcp 0 0 127.0.0.1:631 0.0.0.0:* LISTEN 1764/cupsd
tcp 0 0 127.0.0.54:53 0.0.0.0:* LISTEN 678/systemd-resolve
tcp 0 0 127.0.0.53:53 0.0.0.0:* LISTEN 678/systemd-resolve
tcp 0 0 0.0.0.0:22 0.0.0.0:* LISTEN 1/systemd
tcp6 0 0 :::1:631 :::* LISTEN 1764/cupsd
tcp6 0 0 :::22 :::* LISTEN 1/systemd
udp 0 0 0.0.0.0:5353 0.0.0.0:* 1428/avahi-daemon:
udp 0 0 10.0.2.15:3702 0.0.0.0:* 6536/python3
udp 0 0 239.255.255.250:3702 0.0.0.0:* 6536/python3
udp 0 0 127.0.0.54:53 0.0.0.0:* 678/systemd-resolve
udp 0 0 127.0.0.53:53 0.0.0.0:* 678/systemd-resolve
udp 0 0 127.0.0.1:323 0.0.0.0:* 1890/chronyd
udp 0 0 0.0.0.0:56006 0.0.0.0:* 6536/python3
udp6 0 0 :::5353 :::* 1428/avahi-daemon:
udp6 0 0 fe80::a00:27ff:fe47:d601:3702 :::* 6536/python3
udp6 0 0 ff02::c:3702 :::* 6536/python3
udp6 0 0 :::48816 :::* 6536/python3
udp6 0 0 :::1:323 :::* 1890/chronyd

```

- What I learned:

I learned how to identify a Linux system's hostname, IP address, routing table, and active network services. Network interface and default gateway were verified using *ip a* and *ip r*. Network configuration was further verified using *hostnamectl*. I also used *ss* and *netstat* to review listening ports and verify that network services like SSH were operating correctly. Open ports included SSH (22), DNS (53), and several local system services.

CHALLENGES:

Ubuntu 26.04 did not include the traditional "last" utility by default. The system recommended installing the *wtmpdb* package to access login history functionality. After researching the issue, I installed the recommended package and verified successful functionality using the *last* command. I learned that Linux distributions may package administrative tools differently across versions.

OBJECTIVE: Capture and analyze live network traffic to identify protocols, understand packet-level communications, and practice network troubleshooting

ENVIRONMENT:

Ubuntu 26.04 VM running in VirtualBox

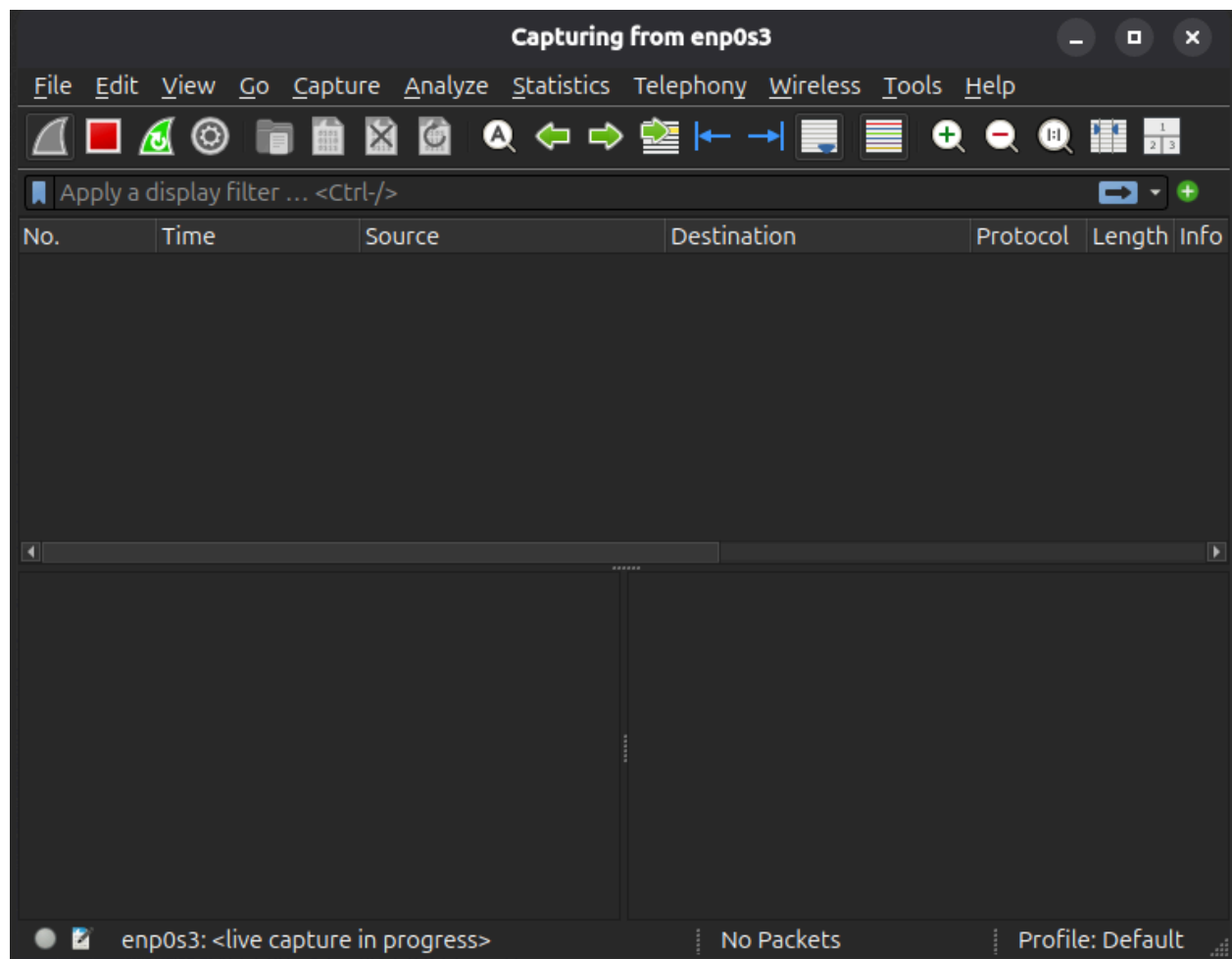
Wireshark Version: 4.6.4

Interface: enp0s3

VM IP: 10.0.2.15

Gateway: 10.0.2.2

```
user@Ubuntu:~$ groups $USER
user : user adm cdrom sudo dip plugdev users lpadmin lxd wireshark
```



SECTION 1: DNS TRAFFIC

- Commands that generated traffic:
nslookup google.com
nslookup github.com
nslookup youtube.com
dig amazon.com
- Filter used:
dns
- DNS server IP observed:
127.0.0.53
- What a query/response pair looks like:

Time	Source	Destination	Protocol	Length	Info
11 9.186234600	10.0.2.15	192.168.1.1	DNS	100	Standard query 0xbc13 A connectivity-check.ubuntu.com OPT
12 9.189844207	192.168.1.1	10.0.2.15	DNS	196	Standard query response 0xbc13 A connectivity-check.ubuntu.com OPT
25 22.103243836	10.0.2.15	192.168.1.1	DNS	81	Standard query 0x3f96 A github.com OPT
26 22.107397330	192.168.1.1	10.0.2.15	DNS	97	Standard query response 0x3f96 A github.com A 140.82.113.4 OPT
27 22.109293202	10.0.2.15	192.168.1.1	DNS	81	Standard query 0x5f52 AAAA github.com OPT
28 22.112786871	192.168.1.1	10.0.2.15	DNS	146	Standard query response 0x5f52 AAAA github.com S0A dns1.p08
29 29.135000027	10.0.2.15	192.168.1.1	DNS	82	Standard query 0x5738 A youtube.com OPT
30 29.137800712	192.168.1.1	10.0.2.15	DNS	98	Standard query response 0x5738 A youtube.com A 142.250.68.20
31 29.139138092	10.0.2.15	192.168.1.1	DNS	82	Standard query 0x8f0d AAAA youtube.com OPT
32 29.141761946	192.168.1.1	10.0.2.15	DNS	110	Standard query response 0x8f0d AAAA youtube.com AAAA 2607:f8
33 32.481618013	10.0.2.15	192.168.1.1	DNS	81	Standard query 0x1a11 A amazon.com OPT
34 32.484945409	192.168.1.1	10.0.2.15	DNS	129	Standard query response 0x1a11 A amazon.com A 98.82.161.185
59 116.551804883	10.0.2.15	192.168.1.1	DNS	81	Standard query 0x5622 A google.com OPT
60 116.552648123	192.168.1.1	10.0.2.15	DNS	97	Standard query response 0x5622 A google.com A 142.250.72.14
61 116.554313465	10.0.2.15	192.168.1.1	DNS	81	Standard query 0x2096 AAAA google.com OPT
62 116.557110571	192.168.1.1	10.0.2.15	DNS	109	Standard query response 0x2096 AAAA google.com AAAA 2607:f8

- What I learned:

I learned how DNS translates domain names into IP addresses used for network communication. I used *nslookup* and *dig* to observe DNS request and response packets. An interesting observation was that Ubuntu 26.04 used 127.0.0.53 as its DNS server, which is the local systemd-resolved service rather than a public DNS server. The packet capture also showed that DNS primarily operates over UDP port 53 using a request/response communication model. I also learned that a DNS query packet contains the request domain name, and the response packet contains the corresponding IP address returned by the DNS resolver. Query and response packets share the same transaction ID, which makes it possible to match requests with their responses.

SECTION 2: TCP HANDSHAKE

- Commands that generated traffic:
curl https://neverssl.com (failed)
curl https://google.com
- Filter used:
tcp.flags.syn == 1
- SYN/SYN-ACK/ACK sequence observed:

Time	Source	Destination	Protocol	Length	Info
702 573.288171170	10.0.2.15	142.250.217.142	TCP	74	35314 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM
703 573.292738446	142.250.217.142	10.0.2.15	TCP	60	443 → 35314 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
704 573.292762803	10.0.2.15	142.250.217.142	TCP	54	35314 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=0
705 573.294110169	10.0.2.15	142.250.217.142	TLSv1.3	1624	Client Hello (SNI=google.com)

- Port observed:

Source Port: 35314

Destination Port: 443

- What I learned:

I learned that the TCP three-way handshake establishes a reliable connection between a client and server before data is exchanged. The client initiates communication with a SYN packet. The server responds with a SYN-ACK packet, and the client completes the process with an ACK packet. I also learned that the source port was an ephemeral port chosen by the client, while port 443 is the standard port used for HTTPS traffic.

Observing the handshake provided insight into how network sessions are established before application data is transmitted.

SECTION 3: HTTPS/TLS TRAFFIC

- Commands that generated traffic:

`curl https://google.com`

`curl https://github.com`

- Filter used:

`tls`

- Client Hello and Server Hello observed:

1169	2104.7901630...	10.0.2.15	142.250.72.14	TLSv1.3	1624 Client Hello (SNI=google.com)
1172	2104.8007854...	142.250.72.14	10.0.2.15	TLSv1.3	1494 Server Hello, Change Cipher Spec
1178	2104.8010338...	142.250.72.14	10.0.2.15	TLSv1.3	550 Application Data
1224	2117.4274174...	10.0.2.15	140.82.113.3	TLSv1.3	1624 Client Hello (SNI=github.com)
1227	2117.4378348...	140.82.113.3	10.0.2.15	TLSv1.3	1494 Server Hello, Change Cipher Spec, Application Data
1231	2117.4380947...	140.82.113.3	10.0.2.15	TLSv1.3	278 Application Data, Application Data, Application Data

- What I learned:

I learned how TLS secures communications between a client and server by encrypting data transmitted across the network. The packets allowed me to see how encryption is negotiated before application data is exchanged. Although connection details like IP addresses, ports, and protocol information were visible, the actual webpage content remained unreadable. This is because it was encrypted by TLS. I also observed that HTTPS traffic commonly operates over TCP port 443 and relies on TLS to secure communications.

SECTION 4: SSH TRAFFIC

- Commands that generated traffic:

`ssh localhost`

`hostname -I`

`ssh user@10.0.2.15`

`sudo ss -tnp | grep :22`

- Filter used:

`tcp.port == 22`

- Port 22 traffic observed:

```
user@Ubuntu:~$ sudo ss -tnp | grep :22
[sudo: authenticate] Password:
ESTAB 0      0      127.0.0.1:42132    127.0.0.1:22      users:(("ssh",pid=8010,fd=3))
ESTAB 0      0      10.0.2.15:38922   10.0.2.15:22      users:(("ssh",pid=8308,fd=3))
ESTAB 0      0      10.0.2.15:39158   10.0.2.15:22      users:(("ssh",pid=8187,fd=3))
ESTAB 0      36      127.0.0.1:22      127.0.0.1:42132   users:(("sshd-session",pid=8080,fd=9),("sshd-session",pid=8011,fd=9))
ESTAB 0      36      10.0.2.15:22      10.0.2.15:39158   users:(("sshd-session",pid=8224,fd=9),("sshd-session",pid=8188,fd=9))
ESTAB 0      36      10.0.2.15:22      10.0.2.15:38922   users:(("sshd-session",pid=8345,fd=9),("sshd-session",pid=8309,fd=9))
```

SSH traffic on port 22 was confirmed via ss, but was not visible in Wireshark on *enp0s3*.

- How this connects to the Linux lab:

```
user@Ubuntu:~$ ssh user@10.0.2.15
user@10.0.2.15's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-22-generic x86_64)

* Documentation:  https://docs.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Sun Jun 14 05:04:27 2026 from 10.0.2.15
```

- What I learned:

I learned how SSH provides secure remote access to Linux systems using encrypted communications over TCP port 22. Active SSH connections were verified using the ss utility, which confirmed that the SSH service was listening and accepting connections. SSH encrypts authentication credentials, commands, and session data before transmission. Telnet transmits all information in plaintext, which makes usernames, passwords, and commands vulnerable to interception. SSH is the industry standard for remote administration because of its encryption and authentication capabilities. I also observed that when a system connects to itself, some traffic will not appear on the monitored network interface. This activity directly validated the OpenSSH configuration completed during the Linux section, confirming that the service was installed and enabled.

SECTION 5: ICMP TRAFFIC

- Commands that generated traffic:
ping google.com -c 6
- Filter used:
icmp

- Request/Reply pairs observed:

Time	Source	Destination	Protocol	Length	Info
495.662167809	10.0.2.15	34.223.124.45	ICMP	98	Echo (ping) request id=0x3bf3, seq=1/256, ttl=64 (no respons...
496.756209296	10.0.2.15	34.223.124.45	ICMP	98	Echo (ping) request id=0x3bf3, seq=2/512, ttl=64 (no respons...
497.998129412	10.0.2.15	34.223.124.45	ICMP	98	Echo (ping) request id=0x3bf3, seq=3/768, ttl=64 (no respons...
498.998743738	10.0.2.15	34.223.124.45	ICMP	98	Echo (ping) request id=0x3bf3, seq=4/1024, ttl=64 (no respon...
4343.6598170...	10.0.2.15	142.250.72.14	ICMP	98	Echo (ping) request id=0x3bf4, seq=1/256, ttl=64 (reply in 2...
4344.6644475...	10.0.2.15	142.250.72.14	ICMP	98	Echo (ping) request id=0x3bf4, seq=2/512, ttl=64 (reply in 2...
4345.6661137...	10.0.2.15	142.250.72.14	ICMP	98	Echo (ping) request id=0x3bf4, seq=3/768, ttl=64 (reply in 2...
4346.8286301...	10.0.2.15	142.250.72.14	ICMP	98	Echo (ping) request id=0x3bf4, seq=4/1024, ttl=64 (reply in ...
4348.2822569...	10.0.2.15	142.250.72.14	ICMP	98	Echo (ping) request id=0x3bf4, seq=5/1280, ttl=64 (reply in ...
4349.3346510...	10.0.2.15	142.250.72.14	ICMP	98	Echo (ping) request id=0x3bf4, seq=6/1536, ttl=64 (reply in ...
4343.6636946...	142.250.72.14	10.0.2.15	ICMP	98	Echo (ping) reply id=0x3bf4, seq=1/256, ttl=64 (request in...
4344.6682523...	142.250.72.14	10.0.2.15	ICMP	98	Echo (ping) reply id=0x3bf4, seq=2/512, ttl=64 (request in...
4345.6704425...	142.250.72.14	10.0.2.15	ICMP	98	Echo (ping) reply id=0x3bf4, seq=3/768, ttl=64 (request in...
4346.8325871...	142.250.72.14	10.0.2.15	ICMP	98	Echo (ping) reply id=0x3bf4, seq=4/1024, ttl=64 (request i...
4348.2858988...	142.250.72.14	10.0.2.15	ICMP	98	Echo (ping) reply id=0x3bf4, seq=5/1280, ttl=64 (request i...
4349.3393808...	142.250.72.14	10.0.2.15	ICMP	98	Echo (ping) reply id=0x3bf4, seq=6/1536, ttl=64 (request i...

- What I learned:

I learned that ICMP is used for network diagnostics and connectivity testing. The ping command generated Echo Request packets that were sent to the destination host. The Echo Reply packets confirmed successful communication. I was able to verify that the destination system was reachable and responding normally. I also learned that ICMP does not establish a connection like TCP and instead operates as a lightweight protocol tool for troubleshooting.

SECTION 6: TROUBLESHOOTING EXERCISE

- Problem observed:

DNS resolution failed for the domain *fake-domain-12345.com*. This prevents the system from resolving the hostname to an IP address.

- Why was the DNS layer investigated first:

Hostname resolution must occur before network communication can take place.

- Investigation steps:

Ran *nslookup fake-domain-12345.com*

Ran *dig fake-domain-12345.com*

Ran *ping fake-domain-12345.com*

Reviewed DNS packets in Wireshark

Verified normal DNS functionality with *nslookup google.com*

Reviewed DNS resolver configuration using *cat /etc/resolv.conf*

- Wireshark findings:

Multiple DNS queries for *fake-domain-12345.com* were observed. The DNS server responded with "No such name" responses instead of returning an IP address.

- Root cause:

The requested domain did not exist in DNS and therefore could not be resolved to an IP address.

- What NXDOMAIN means:

Non-Existent Domain indicates that the requested domain name does not exist in DNS.

- How Wireshark helped vs command line alone:

Wireshark provided direct visibility into the DNS query and response packets. This allows the root cause to be identified immediately instead of relying only on application error messages.

5230.9658125...	192.168.1.1	10.0.2.15	DNS	165	Standard query response	0x79ed	No such name	A fake-domain-123...
5240.0494287...	192.168.1.1	10.0.2.15	DNS	92	Standard query response	0x06ad	No such name	A fake-domain-123...
5260.3302551...	192.168.1.1	10.0.2.15	DNS	92	Standard query response	0x04fc	No such name	A fake-domain-123...
5260.3346917...	192.168.1.1	10.0.2.15	DNS	165	Standard query response	0x4ea1	No such name	AAAA fake-domain-...
5260.3356043...	192.168.1.1	10.0.2.15	DNS	92	Standard query response	0xf966	No such name	A fake-domain-123...
5260.3356043...	192.168.1.1	10.0.2.15	DNS	92	Standard query response	0x593c	No such name	AAAA fake-domain-...
5313.3996055...	10.0.2.15	192.168.1.1	DNS	81	Standard query	0x01e7	A google.com OPT	
5313.4024944...	192.168.1.1	10.0.2.15	DNS	97	Standard query response	0x01e7	A google.com A	142.250.217.142...
5313.4033525...	10.0.2.15	192.168.1.1	DNS	81	Standard query	0xe27c	AAAA google.com OPT	
5313.4059592...	192.168.1.1	10.0.2.15	DNS	109	Standard query response	0xe27c	AAAA google.com AAAA	2607:f8b0...
5319.2202470...	10.0.2.15	192.168.1.1	DNS	100	Standard query	0xbfac	AAAA connectivity-check.ubuntu.com OPT	
5319.2231060...	192.168.1.1	10.0.2.15	DNS	268	Standard query response	0xbfac	AAAA connectivity-check.ubuntu...	
5409.2031742...	10.0.2.15	192.168.1.1	DNS	100	Standard query	0x3930	A connectivity-check.ubuntu.com OPT	
5409.2065726...	192.168.1.1	10.0.2.15	DNS	196	Standard query response	0x3930	A connectivity-check.ubuntu.co...	
5619.4995533...	10.0.2.15	192.168.1.1	DNS	100	Standard query	0xce45	AAAA connectivity-check.ubuntu.com OPT	
5619.5023156...	192.168.1.1	10.0.2.15	DNS	268	Standard query response	0xce45	AAAA connectivity-check.ubuntu...	
5710.4745294...	10.0.2.15	192.168.1.1	DNS	100	Standard query	0x7add	A connectivity-check.ubuntu.com OPT	
5710.4775456...	192.168.1.1	10.0.2.15	DNS	196	Standard query response	0x7add	A connectivity-check.ubuntu.co...	
5718.7923925...	10.0.2.15	192.168.1.1	DNS	84	Standard query	0xd131	A cdn.fwupd.org OPT	
5718.7925854...	10.0.2.15	192.168.1.1	DNS	84	Standard query	0x2cdb	AAAA cdn.fwupd.org OPT	
5718.7955260...	192.168.1.1	10.0.2.15	DNS	151	Standard query response	0xd131	A cdn.fwupd.org CNAME dualstac...	
5718.7964548...	192.168.1.1	10.0.2.15	DNS	163	Standard query response	0x2cdb	AAAA cdn.fwupd.org CNAME duals...	

```

user@Ubuntu:~$ nslookup fake-domain-12345.com
Server:      127.0.0.53
Address:     127.0.0.53#53

** server can't find fake-domain-12345.com: NXDOMAIN

user@Ubuntu:~$ dig fake-domain-12345.com

; <<>> DiG 9.20.18-1ubuntu2.1-Ubuntu <<>> fake-domain-12345.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 59694
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:: udp: 65494
;; QUESTION SECTION:
;fake-domain-12345.com.      IN      A

;; Query time: 2 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Sun Jun 14 05:41:19 UTC 2026
;; MSG SIZE rcvd: 50

user@Ubuntu:~$ ping fake-domain-12345.com
ping: fake-domain-12345.com: Name or service not known

```

```

user@Ubuntu:~$ cat /etc/resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search .

```

- What I learned:

I learned how packet analysis can be used to troubleshoot DNS-related issues. It can be used to identify the exact point of failure in the communication process. The Wireshark capture showed DNS queries being sent successfully and NXDOMAIN responses being returned by the DNS server. Comparing the failed domain with a successful query helped verify that the issue was specifically the requested domain. I also learned that packet analysis can assist in finding underlying issues with application errors like a failed ping.

OVERALL FINDINGS:

- Protocols observed:

DNS, TCP, TLS/HTTPS, SSH, and ICMP traffic were captured and analyzed during this lab. These demonstrated how systems perform name resolution, establish network connections, secure communications, provide remote access, and test network connectivity.

- Most interesting capture:

The most interesting capture was during the troubleshooting exercise involving the NXDOMAIN response. It demonstrated a real-world investigation workflow. I was able to identify the root cause of a failed domain lookup by analyzing DNS packets in Wireshark.

- How Wireshark connects to the Linux lab:

Wireshark provided packet-level visibility into many of the Linux networking concepts that were explored. DNS lookups, SSH connections, routing, and network troubleshooting could all be observed directly in packet captures.

CHALLENGES:

During the TCP traffic capture phase, the *curl* command was not installed by default on Ubuntu 20.04. The issue was resolved by installing *curl* using *sudo apt install curl -y*. The *curl* command was then verified, and HTTP and HTTPS requests could be generated successfully.

The original exercise used *curl https://neverssl.com* to generate plain HTTP traffic. Even though DNS resolution for the domain worked correctly, the connection to port 80 failed. To troubleshoot this, DNS resolution was verified using *nslookup* and connectivity was tested using *ping*. HTTPS access was then confirmed using *curl https://google.com*. HTTPS traffic was used instead to observe the TCP three-way handshake successfully.

During the SSH analysis phase, using the filter *tcp.port == 22* did not show any packets on the monitored *enp0s3* interface. Active connections were reviewed using *sudo ss -tnp | grep :22*, which confirmed that SSH sessions were active. Further analysis showed that the Ubuntu VM was connecting to itself using localhost (127.0.0.1) and its assigned IP

address (10.0.2.15). Much of the traffic was handled internally by the operating system rather than traversing the monitored VirtualBox network adapter. This is because the source and destination were the same host. As a result, SSH connections were visible through Linux networking tools but were not captured on the enp0s3 interface in Wireshark.

SKILLS DEMONSTRATED:

- Linux administration
- Service management (systemd)
- User and group administration
- SSH configuration and troubleshooting
- Log analysis
- Network traffic analysis
- DNS troubleshooting
- TCP/IP fundamentals
- TLS/HTTPS analysis
- Packet capture filtering
- Root cause analysis
- Technical documentation